AMERICAN EXPRESS · CODESTREET 2026

The End-to-End Servicing Agent

Resolves fee reversals, credit-limit increases & card replacements in a **single conversation** — with a **tamper-evident audit trail** and a human escape hatch.

Problem Statement: **End-to-End Servicing Agent**

Team: **<your team name>** Members: **<names>**

THE PROBLEM

Routine servicing is slow, costly, and repetitive

- **Fee reversals, limit increases, and replacement cards are the highest-frequency requests —** and the most tedious to service.
- **Members wait on hold, repeat themselves, and get transferred between agents.** Every touch adds compliance overhead.
- **Skilled human agents spend their time on routine work** instead of the complex cases that actually need them.

~$13.50
cost per human-assisted contact

~$1.84
cost per self-service contact

The opportunity: resolve the routine autonomously — safely — and free humans for the hard cases.

OUR SOLUTION

An agent that resolves the request — not just talks about it

Resolves end-to-end

Executes the actual fee waiver, limit change, or card order — start to finish, in one interaction.

Provably auditable

Every decision & action is written to a hash-chained, tamper-evident audit trail — immutable by construction.

Escalates gracefully

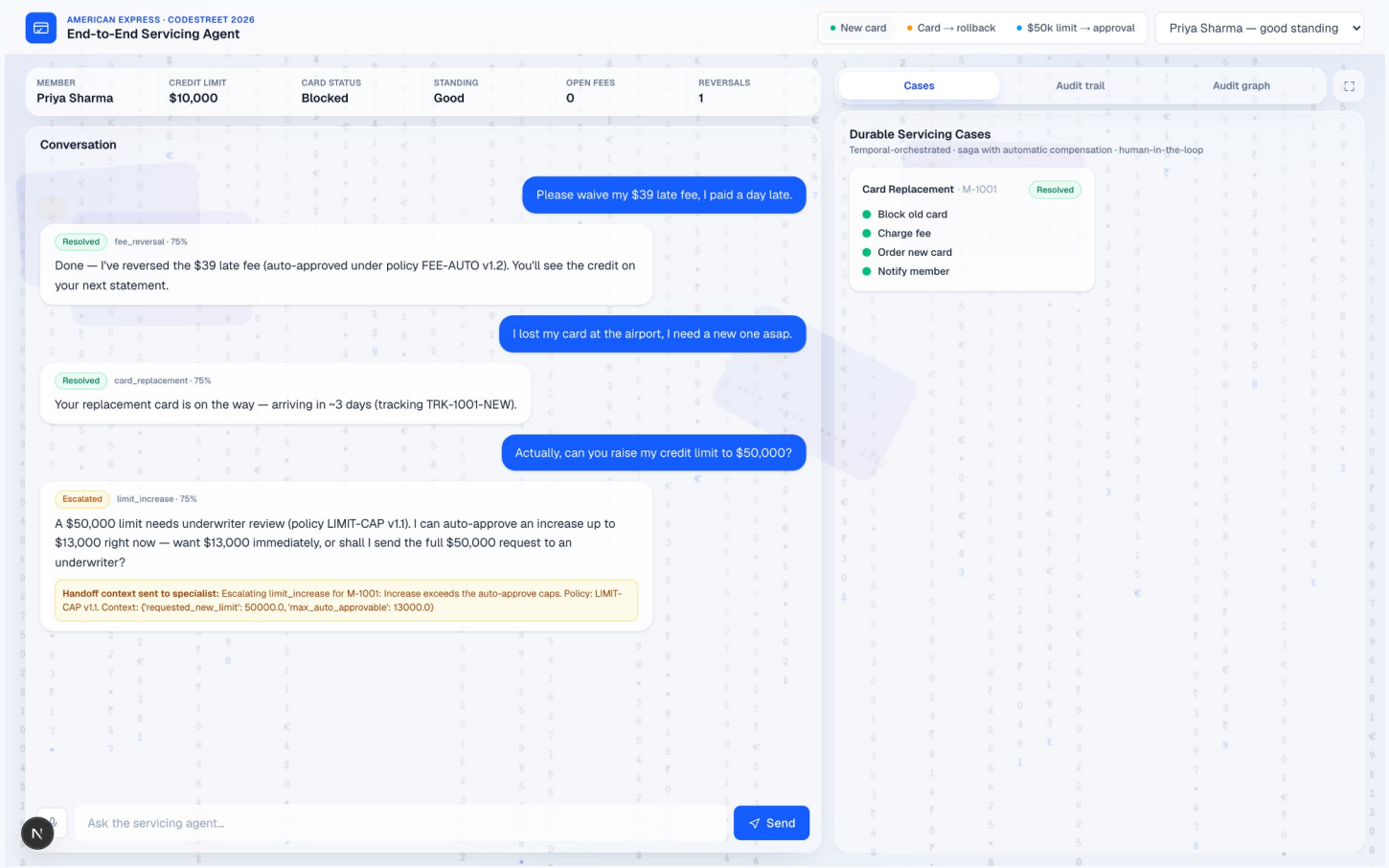
When policy limits or low confidence are hit, hands off to a human with full context — no repeating.

Single interaction. Real actions. Complete traceability.

Plus multi-turn follow-ups (slot-filling) · voice input · safe out-of-scope handling (answer / escalate / clarify)

THE PRODUCT

One console — chat, durable cases & the audit trail



What you're seeing

- **Policy citation in every reply**
  "auto-approved under FEE-AUTO v1.2"
- **Counterfactual on escalation**
  "I can auto-approve up to $13,000 now"
- **Escalation with full handoff context**
  the specialist gets everything
- **Durable cases + live audit / graph**
  one screen, in real time

Live capture of the running app

HOW A REQUEST IS DECIDED

The LLM proposes — a versioned engine decides and cites the rule

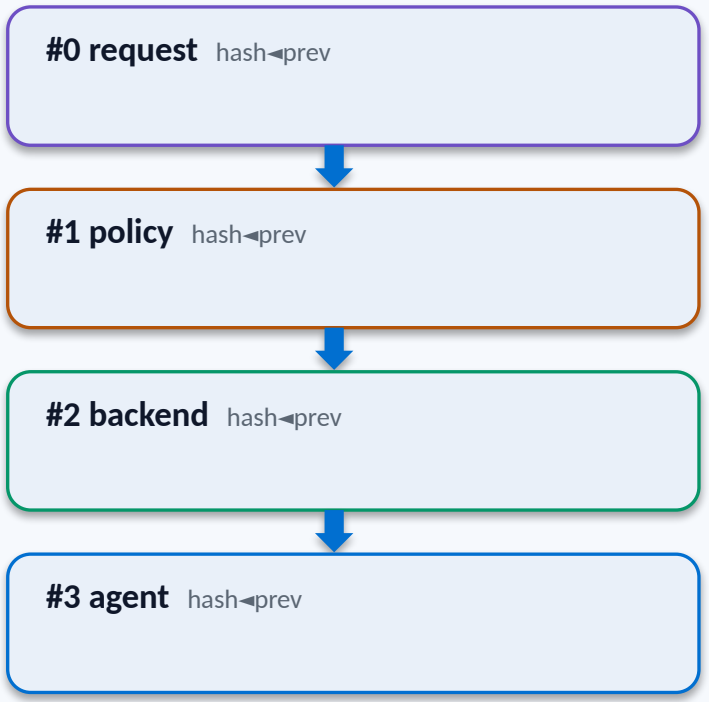
⛓ **Hash-chained audit trail** — every decision sealed in order, tagged with the rule that fired (e.g. LIMIT-CAP v1.1)

The model never has authority over money — a deterministic, versioned rule engine decides and cites the exact rule; a second agent verifies before anything executes.

DIFFERENTIATOR 1

Tamper-evident audit trail — trust by architecture

- **Every decision, policy check & backend call is hash-chained (SHA-256)** — each entry seals the one before it.
- **Edit any past entry and every downstream hash breaks** — caught live in the demo (verify flips to ‘broken @ seq N’).
- **Complete chain-of-custody, not sampling** — the difference between ‘we think’ and ‘we can prove’.
- **Persisted to Neo4j — explorable as an analyst graph** — query the whole chain of custody, not scroll a log.
- **Maps to model-risk (SR 11-7) & adverse-action explainability** — audit-ready by design.



DIFFERENTIATOR 2 · BEYOND THE BRIEF

Servicing cases that can’t be left half-done

- **Every high-stakes request becomes a durable Temporal workflow** — it survives crashes and resumes exactly where it stopped.
- **Saga with automatic compensation** — if a step fails, completed steps roll back in reverse. Never partial.
- **Durable human-in-the-loop approval** — over-policy cases wait on an underwriter for as long as it takes.
- **Live status + replayable event history** — queryable in real time; the log is itself an audit.

Card replacement saga

✓ block old card

✓ charge fee

✗ order fulfillment — fails

↩ refund fee (compensate)

↩ unblock card (compensate)

= rolled back · consistent

DIFFERENTIATOR 3

Trustworthy by construction — the LLM never decides

Policy-as-Code

Versioned declarative rules decide; the LLM only proposes. Every decision cites the exact rule + version (e.g. LIMIT-CAP v1.1). Change policy = edit data + bump a version, no agent code.

Self-verifying agent

A second reviewer LLM must agree the interpretation matches the request before acting — a bounded propose → verify → revise loop; escalates if the two agents can't agree. Risk-based, so it's spent where it matters.

Counterfactual answers

On a denial we compute the nearest-approvable outcome ('I can do $15,600 now'), mapping to ECOA/CFPB adverse-action 'specific reasons'.

Decision provenance

Rules become nodes in Neo4j — analysts query which rule approved or denied across members, not just scroll a log.

EVALUATION

Measured, not asserted

Metric	Result	How it's graded
Intent classification accuracy	**90% (n=20)**	labeled eval set · LLM target ≥95%
First-contact resolution	**80% (12/15)**	multi-turn outcome check
Correct handling (resolve / safe-escalate)	**86.7%**	outcome check, in-scope
Policy violations	**0**	engine caps enforced + verified
Audit integrity	**100% intact**	hash-chain verification

We report TRUE resolution + escalation rate — not just deflection. Financial-services deflection is realistically 25–45%; finance teams target <1% error, because one wrong answer is a compliance event.

WHY AMEX · WHY NOW

The governance layer that makes autonomy deployable

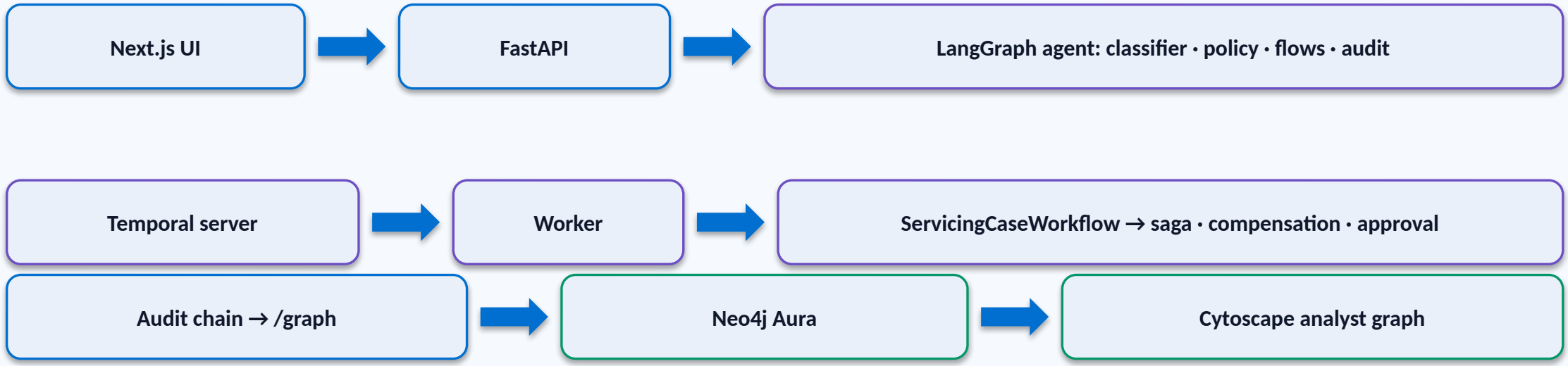
- **Amex's 2026 direction is agentic commerce — agents that decide and act** — this is exactly that, applied to servicing.
- **Only 21% of enterprises have mature agentic-AI governance, yet 74% expect to use AI agents by 2027** (Deloitte) — most are scaling agents without the guardrails.
- **Deloitte names the missing controls: audit trails of every agent action + human-approval boundaries** — the exact two things we built.

> **"With every use case, we've ensured there is human oversight."** — Ravi Radhakrishnan, EVP & CIO, American Express (Forbes, 2025)

Sources: Deloitte, "Agentic AI is scaling faster than guardrails" (2025) · Forbes / Peter High, "Ravi Radhakrishnan on Driving AI Innovation at American Express" (May 2025)

ARCHITECTURE

Built to be real

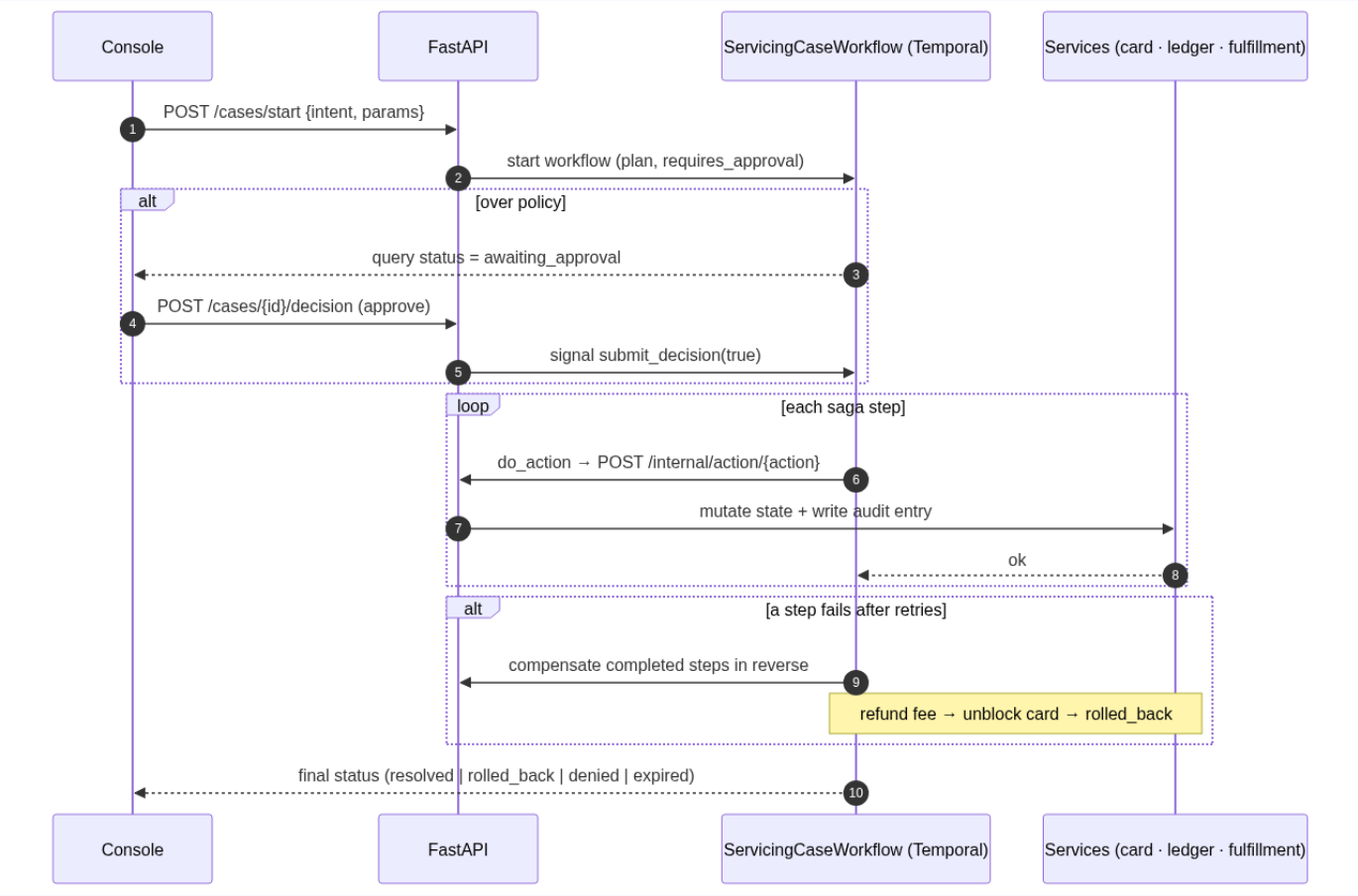


Stack Python · FastAPI · LangGraph · Gemini · Temporal · Neo4j · Next.js · Cytoscape · SHA-256 hash-chain

Core-banking is mocked today; each call swaps to a real Amex API with no change to the agent logic.

SYSTEM DESIGN

Low-level design — how a durable case executes



Components

FastAPI owns card state + audit chain

LangGraph classify · policy · flows · assist

Temporal worker durable saga + compensation

Neo4j Aura audit graph for analysts

Audit entry (hash-chained)

```
{ seq, actor, action,
  payload, prev_hash, hash }
```

Key APIs

```
POST /chat
POST /cases/start · /decision
POST /internal/action/{action}
GET  /cases · /graph
```

WHAT'S NEXT

From prototype to production

- **Round 2: integrate real servicing APIs, add voice, expand intents,** and add an LLM-judge clarity eval.
- **Production audit store: append-only, write-once storage behind the same hash-chain** — exportable for auditors.
- **Guardrails & governance: per-action risk tiers, kill-switch, policy versioning** — the governance substrate for Amex agents.

The ask: advance us to the prototype round — the core already works end-to-end.